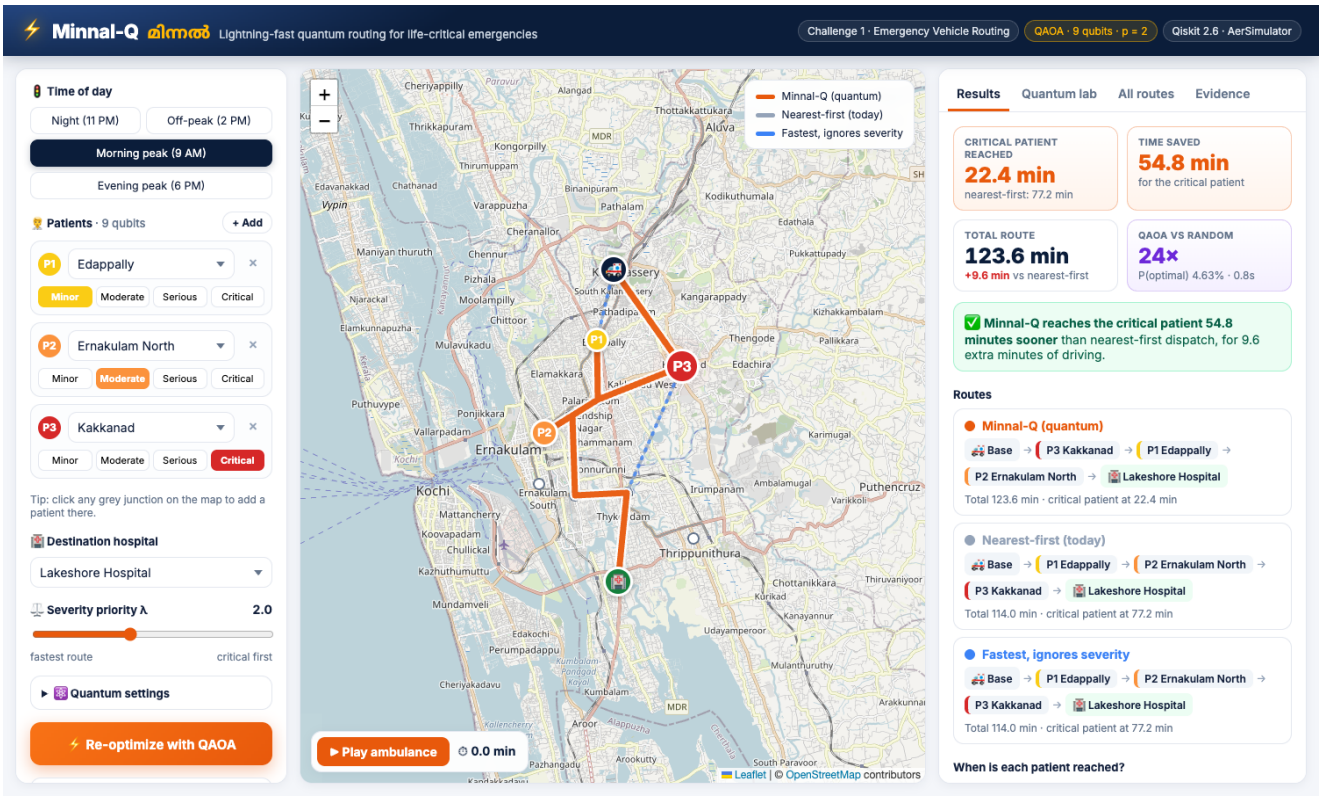
Minnal-Q

മിന്നൽ

Study Guide

How the problem is solved · What is innovative · How the quantum parts work



Section	What you will learn
1. The problem	What we solve, for whom, and why it is hard
2. The solution in 7 steps	The full pipeline from map to answer
3. Worked example	Every number in the Kochi demo, by hand
4. Quantum components	Qubits, QUBO, Ising, QAOA circuit, hybrid training, sampling, metrics
5. Innovation	What is new compared with the 14 reference projects
6. Code map	Which file does what, and how a request flows
7. Results and limits	Measured numbers and honest limitations
8. Judge questions	Likely questions with model answers
9. Glossary + cheat sheet	Key terms and numbers to memorise

1. The problem

One-line summary: when one ambulance must reach several injured people, Minnal-Q decides **the order of the stops** so that the most critical patients are reached first, using real time-of-day traffic, and it solves that ordering with a quantum algorithm (QAOA).

1.1 The situation

In a mass-casualty event (a bus crash on NH-66, flooding, a building collapse), one ambulance may have to collect patients from several places before reaching a hospital. Dispatchers usually go to the **nearest** patient first. That keeps driving short, but a **critically injured patient can be left waiting** while minor cases are picked up. Kochi's traffic also changes through the day, so the best order at 9 a.m. is not the best order at 11 p.m.

1.2 What exactly we optimise

- **Decision:** the visiting order of n patients. The ambulance starts at its base (Kalamassery) and ends at a hospital.
- **Goal 1:** keep total travel time low, using traffic-adjusted road times.
- **Goal 2:** reach high-severity patients early. Severity 1–4 maps to weights 1, 2, 4, 8.
- **Trade-off knob λ:** λ = 0 means "fastest route only"; a larger λ means "critical patients first".

1.3 Why it is hard

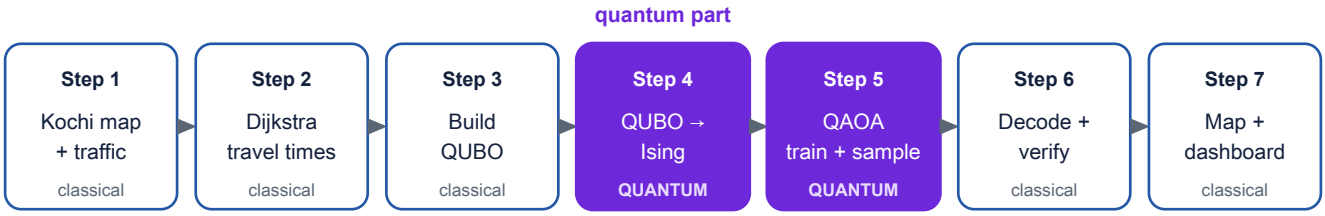
Patients (n)	Possible orders (n!)	Qubits for Minnal-Q (n²)
3	6	9
4	24	16
6	720	36
10	3,628,800	100
15	≈ 1.3 trillion	225

The number of orders grows factorially. Severity and time-dependent traffic also break simple rules like "nearest first". This is a **combinatorial optimisation** problem, which is exactly the class QAOA is designed for.

1.4 Who benefits

Critical patients (trauma, cardiac arrest, stroke) for whom minutes matter; ambulance crews and dispatchers; hospital emergency teams. SDG 3 (Good Health) and SDG 11 (Sustainable Cities).

2. The solution in 7 steps



Files: city.py → classical.py / qubo.py → qaoa.py → server.py → web/

Step	What happens	Type	File
1	Build the Kochi road graph: 15 places, 22 roads. Each road's minutes × the traffic factor for the chosen time slot.	Classical	city.py
2	Dijkstra's shortest paths give the travel time between every pair of stops (the matrix T).	Classical	city.py
3	Write the routing problem as a QUBO: a cost formula over n² yes/no variables.	Classical	qubo.py

Step	What happens	Type	File
4	Convert the QUBO into an Ising Hamiltonian (Z and ZZ Pauli terms) as a SparsePauliOp.	**Quantum**	qubo.py
5	Build the QAOA circuit, train its angles with COBYLA (hybrid loop), then sample 4,096 shots on AerSimulator.	**Quantum**	qaoa.py
6	Decode bitstrings into routes, discard invalid ones, keep the best. Verify against brute force.	Classical	qaoa.py, classical.py
7	Return everything as JSON and draw it: map, animated ambulance, charts, evidence.	Classical	server.py, web/

3. Worked example: the Kochi demo, by hand

Morning peak (9 AM), λ = 2. The ambulance starts at **Kalamassery** and ends at **Lakeshore Hospital**.

3.1 Inputs

Patient	Location	Severity	Weight w
P1	Edappally	1 · Minor	1
P2	Ernakulam North	2 · Moderate	2
P3	Kakkanad	4 · Critical	8

3.2 Traffic multipliers (synthetic, by road type)

Time slot	Highway	Arterial	City roads
Night (11 PM)	0.8×	0.8×	0.8×
Off-peak (2 PM)	1.0×	1.1×	1.2×
Morning peak (9 AM)	**2.0×**	**1.6×**	**1.5×**
Evening peak (6 PM)	1.6×	1.5×	2.1×

Example: Base → Kakkanad is an arterial road of 14 min off-peak. At morning peak: 14 × 1.6 = **22.4 min**.

3.3 Travel-time matrix T (minutes, after Dijkstra)

From \ To	Base	P1 Edappally	P2 Ern. North	P3 Kakkanad	Lakeshore
Base	–	16.0	43.0	22.4	59.2
P1 Edappally	16.0	–	27.0	31.2	46.0
P2 Ernakulam North	43.0	27.0	–	34.2	43.0
P3 Kakkanad	22.4	31.2	34.2	–	36.8

3.4 Scoring all 6 possible orders

Cost = total time + λ × Σ (weight × position), with positions 0, 1, 2. Example for the winner P3 → P1 → P2: total = 22.4 + 31.2 + 27.0 + 43.0 = 123.6; position penalty = 8·0 + 1·1 + 2·2 = 5; cost = 123.6 + 2 × 5 = **133.6**.

Order	Total time	Position penalty	Cost (λ = 2)	Critical patient reached	Note
P3 → P1 → P2	123.6	5	**133.6**	**22.4 min**	**Minnal-Q optimum**
P3 → P2 → P1	129.6	4	137.6	22.4 min	
P1 → P3 → P2	124.4	12	148.4	47.2 min	
P1 → P2 → P3	**114.0**	18	150.0	77.2 min	Nearest-first and fastest
P2 → P1 → P3	138.0	17	172.0	101.2 min	
P2 → P3 → P1	154.4	10	174.4	77.2 min	

The key insight: the fastest route (114.0 min) visits the critical patient **last**, at 77.2 min. Minnal-Q's optimum drives 9.6 minutes more (123.6) but reaches the critical patient at **22.4 min**, **54.8 minutes sooner**. QAOA finds this optimum.

4. The quantum components, step by step

Where quantum is used: only in deciding the visiting order (steps 4–5). Everything else prepares the input or checks the output. If you remove the quantum step, Minnal-Q has no route, so the circuit is meaningful, not decorative.

4.1 From yes/no decisions to qubits

Variable $x_{i,p} = 1$ means "patient i is visited at stop p". Qubit index $k = i \cdot n + p$. With 3 patients that is $3 \times 3 =$ **9 qubits**. The optimal route P3 → P1 → P2 lights up these qubits:

	Stop 1	Stop 2	Stop 3
P1 Edappally (Minor)	q0	**q1 = 1**	q2
P2 Ernakulam North (Moderate)	q3	q4	**q5 = 1**
P3 Kakkanad (Critical)	**q6 = 1**	q7	q8

Bits q0…q8 = **0 1 0 · 0 0 1 · 1 0 0**. Qiskit prints bitstrings in reverse (q8 first), so the measured string is `001100010`. A valid route has exactly one 1 in every row and every column (a permutation matrix). Only 6 of the $2^9 = 512$ bitstrings are valid.

4.2 The QUBO cost function

$$H(x) = A \cdot \Sigma_i (1 - \Sigma_p x_{i,p})^2 + A \cdot \Sigma_p (1 - \Sigma_i x_{i,p})^2 + \Sigma_i T_{S,i} x_{i,0} + \Sigma_{i \neq j} \Sigma_p T_{i,j} x_{i,p} x_{j,p+1} + \Sigma_i T_{i,H} x_{i,n-1} + \lambda \cdot \Sigma_i \Sigma_p w_i \cdot p \cdot x_{i,p}$$

Term	Meaning	Type	Demo value
Constraint penalty (A)	Adds a big cost if a patient is skipped or visited twice, or a stop is empty or doubled	linear + pairwise	A = 59.2
Start leg $T_{S,i}$	Base → first patient	linear	e.g. 22.4 to Kakkanad
Middle legs $T_{i,j}$	Paid only if j comes right after i ($x_{i,p} \cdot x_{j,p+1}$)	pairwise (quadratic)	e.g. 31.2 Kakkanad→Edappally
End leg $T_{i,H}$	Last patient → hospital	linear	e.g. 43.0 Ern. North→Lakeshore
Severity λ·w·p	Penalises late visits to severe patients	linear	λ = 2, w ∈ {1, 2, 4, 8}

For every valid route the QUBO energy equals the cost from Section 3 exactly (a tested property). **Why a position penalty?** Exact "severity × arrival time" would need terms of degree 3 or more, which a QUBO cannot hold. λ·w·p is the quadratic stand-in, and arrival times are always recomputed exactly.

4.3 Choosing the penalty A automatically

If A is too small, the lowest-energy state might be an invalid route. If A is too large, the cost landscape becomes flat and QAOA struggles. Minnal-Q starts A at the longest travel leg (59.2 min) and grows it by 1.5× until an **exhaustive check over all 2^N bitstrings** confirms that the QUBO's lowest energy equals the brute-force optimum. For the demo, the first value already works: A = 59.2.

4.4 QUBO → Ising Hamiltonian

Quantum hardware measures Pauli-Z, whose values are +1 and −1, while QUBO variables are 0 and 1. Substitute **x = (1 − Z)/2**:

- linear term $c \cdot x \to c/2 - (c/2) \cdot Z$
- pairwise term $c \cdot x_a x_b \to c/4 \cdot (1 - Z_a - Z_b + Z_a Z_b)$
- Result: $H =$ constant $+ \Sigma\, h_k Z_k + \Sigma\, J_{kl} Z_k Z_l$. Demo: **39 Pauli terms**, constant 584.2.
- Built as `qiskit.quantum_info.SparsePauliOp`. A test checks its diagonal equals the QUBO energy on all 512 bitstrings.

4.5 The QAOA circuit

Built with `qiskit.circuit.library.qaoa_ansatz(cost_operator, reps=p)`. The operator is first divided by its largest coefficient so the angles stay in a small, trainable range.

Layer	Gates	What it does
Start	H on every qubit	Equal superposition of all 2^N bitstrings, so every route is possible
Cost layer $U_C(\gamma)$	RZ and RZZ rotations from the Ising terms	Gives each bitstring a phase that depends on its cost
Mixer $U_M(\beta)$	RX on every qubit	Mixes amplitudes so low-cost routes interfere constructively
Repeat	p = 1 or 2 times	More layers concentrate more probability on good routes
Measure	All qubits	Each shot returns one bitstring, which is one candidate route

Demo circuit sizes after compiling to basic gates: p = 1 → depth 18, 30 two-qubit gates; p = 2 → depth 29, 60 two-qubit gates (9 qubits). For 16 qubits: depth 32 / 48 and 84 / 168 two-qubit gates.

4.6 The hybrid training loop

- **Classical side:** SciPy **COBYLA** (a gradient-free optimiser) proposes angles β, γ.
- **Quantum side:** the circuit is evaluated (exact statevector simulation) and returns a score.
- **Score = CVaR, α = 0.2:** the average cost of the best 20% of outcomes. This focuses training on producing some very good samples, instead of a good average. Plain expectation is available as an option.
- **3 random restarts** avoid poor local minima. **Warm start:** p = 2 begins from the best p = 1 angles (γ, γ, β, β).
- The loop stops when COBYLA converges (up to 150 steps per restart). Demo training takes under 1 second.

4.7 Sampling and decoding

- The trained circuit is transpiled for and run on **qiskit_aer.AerSimulator** via `qiskit_aer.primitives.SamplerV2`, with **4,096 shots**.
- Each bitstring is reshaped into an n×n grid. If it is a valid permutation, it becomes a route; otherwise it is counted as invalid and discarded (never hidden).
- The lowest-cost valid route is Minnal-Q's answer. The app also shows how often each route was measured.

4.8 How we measure the quantum part

Metric	Meaning	Demo (3 patients, p = 2)
P(optimal)	Exact probability that one measurement gives an optimal route	**4.63%**
× random	P(optimal) divided by the chance of guessing it (1/512)	**24×**
Valid samples	Share of shots that are valid routes (random: 6/512 = 1.2%)	**24.7%**
Approximation ratio	Optimal cost ÷ average cost of the valid samples (1.0 = perfect)	0.88
Found optimum	Best sampled route equals the brute-force optimum	Yes (all 16 benchmark runs)

4.9 How we know it is correct

- QUBO energy = route cost for every valid route.
- The QUBO's lowest-energy state is a valid route.
- The QUBO's lowest energy = the brute-force optimum (ties allowed).
- Ising operator = QUBO on all 2^N bitstrings.
- QAOA's best sampled route = the exact optimum.
- These five run live in the app's Evidence tab; 22 automated tests (pytest) cover them across all time slots, plus the API.

5. What is innovative

We reviewed 14 earlier hackathon projects. None combines ambulances + multi-stop routing + distance + traffic. Minnal-Q does, and adds several engineering ideas on top.

#	Innovation	Why it matters
1	**Time-of-day traffic inside the QUBO.** Travel times change with the slot (night, off-peak, morning, evening peak).	The quantum problem itself changes with traffic, so the best order can change at 9 a.m. vs 11 p.m.
2	**Severity applied to the visiting order**, not just to which ambulance goes where.	Earlier severity projects solved assignment; Minnal-Q solves routing, which is what Challenge 1 asks.
3	**λ trade-off slider** between "fastest" and "critical first".	Makes the ethics of dispatch visible and tunable.
4	**Auto-verified penalty A.** Exhaustive check that the QUBO's ground state is the true optimum.	Removes a common QUBO bug (wrong penalty gives wrong answers).
5	**CVaR objective + warm-started p = 2.**	Raises P(optimal) from about 2% (p = 1) to 4.6% (p = 2) in the demo.
6	**Live evidence.** Five correctness checks recomputed for every scenario in the app.	Judges can see it is correct, not just told.
7	**Honest metrics.** P(optimal) and valid rate vs random guessing, instead of claiming speed-ups.	Matches the event's transparency rule.
8	**Web app:** click-to-add patients on a Kochi map, animated ambulance, patient timeline.	Makes the result understandable in seconds.
9	**Crash-safe quantum runs** in a large-stack thread, one at a time.	Fixes a real native-code crash when several users connect.

What is **standard** (credited, not claimed as new): the position-based route encoding, QAOA itself, CVaR as a technique, severity weights 1/2/4/8, and the idea of a transparency table.

6. Code map

File	Role
minnal_q/city.py	Kochi nodes (15), roads (22), traffic table, graph builder, Dijkstra travel-time matrix
minnal_q/scenario.py	Patients, severities and weights; exact route evaluation (arrival times, total, critical arrival)
minnal_q/classical.py	Brute force (exact answer), nearest-first greedy, fastest route; the route_cost objective
minnal_q/qubo.py	QUBO builder, auto-penalty, all-energies table, QUBO → Ising (SparsePauliOp), encode/decode
minnal_q/qaoa.py	qaoa_ansatz, COBYLA training (CVaR or expectation), Aer SamplerV2 sampling, all metrics
minnal_q/runtime.py	Runs Qiskit in a 64 MB-stack thread, one job at a time (crash fix)
server.py	FastAPI: GET /api/meta (map, slots, demo); POST /api/optimize (routes, quantum metrics, checks)
web/index.html, style.css, app.js	Leaflet map, controls, animated ambulance, results, Chart.js quantum lab, evidence
run_experiments.py	Benchmark over 2 sizes × 4 slots × p = 1, 2 → results/benchmark.csv
run_ibm.py	Optional: run the 9-qubit circuit on real IBM hardware and save the job ID
tests/	22 tests: QUBO/Ising/QAOA correctness and the web API

Request flow when you press □ Optimize

```
browser (app.js) → POST /api/optimize → server.py validates input → make_scenario (city.py) →
baselines (classical.py) → auto_penalty + to_ising (qubo.py) → solve_qaoa p = 1 then p = 2
```

```
(qaoa.py, in runtime.py's safe thread) → checks → JSON → app.js draws routes, cards, charts.
Results are cached, so repeating a scenario is instant.
```

7. Results and honest limits

7.1 Effect on patients (3 patients, λ = 2)

Time slot	Nearest-first: critical reached	Minnal-Q: critical reached	Saved	Extra driving
Night (11 PM)	36.8 min	**11.2 min**	25.6 min	+6.4 min
Off-peak (2 PM)	51.2 min	**15.4 min**	35.8 min	+8.1 min
Morning peak (9 AM)	77.2 min	**22.4 min**	**54.8 min**	+9.6 min
Evening peak (6 PM)	40.4 min	**21.0 min**	19.4 min	**−0.2 min** (better on both)

7.2 QAOA quality (AerSimulator, CVaR, 3 restarts)

Patients	Qubits	p	Found optimum	P(optimal)	× random	Valid samples	Time
3	9	1	4 / 4	1.1–2.2%	5–11×	6–14%	< 1 s
3	9	2	4 / 4	3.4–4.6%	17–24×	20–25%	< 1 s
4	16	1	4 / 4	0.08–0.19%	49–63×	1–2%	≈ 3 s
4	16	2	4 / 4	0.41–1.24%	266–408×	9–14%	≈ 14 s

7.3 Limitations (say these openly)

- **Simulated:** all results come from Qiskit Aer, with no hardware noise. A real IBM run is optional (run_ibm.py).
- **Synthetic data:** travel times and traffic factors are realistic but invented; place names are real, coordinates approximate.
- **Small sizes:** 3–4 patients per ambulance (9–16 qubits). Brute force is instant at this size.
- **3-patient caveat:** with only 6 valid routes, 4,096 samples will almost surely include the optimum. That is why P(optimal) and "× random" are the meaningful quantum numbers.
- **Proxy:** severity uses a position penalty in the QUBO; real arrival times are recomputed classically.
- **No quantum advantage is claimed**, and the event says it is not expected.

8. Likely judge questions, with model answers

Question	Model answer
Where exactly is quantum used?	Only in choosing the visiting order: QUBO → Ising → QAOA → sampling. The circuit's output is the route.
Is it meaningful or decorative?	Meaningful. Without the circuit there is no answer, and the circuit encodes the real travel and severity costs.
Why QAOA?	Our problem is a QUBO (yes/no variables, pairwise costs). QAOA is designed for QUBOs and uses shallow circuits that suit today's devices.
Why not Grover or VQE?	Grover needs much deeper circuits and an oracle; VQE targets chemistry-style Hamiltonians. QAOA is the natural fit for optimisation.
Is it faster than classical?	No, not at 3–4 patients, and we say so. We show a correct, verified pipeline whose quality we measure against random guessing.
How do you know the answer is right?	Brute force gives the exact optimum; five live checks and 22 tests confirm QUBO, Ising and QAOA agree with it.
Why does QAOA produce invalid routes?	The standard X mixer explores all 2^N bitstrings, and only n! are valid. CVaR and p = 2 raise the valid rate from 1.2% to about 25%; constraint-preserving XY mixers are future work.

Question	Model answer
What is λ?	The severity priority. λ = 0 gives the fastest route; λ = 2 (default) strongly prefers reaching critical patients first.
Why multiple stops for one ambulance?	Mass-casualty and disaster scenarios (bus crash, flood, landslide) where one vehicle collects several patients, or a rapid-response unit doing triage.
How would it scale?	Split patients between ambulances classically, then run one small QAOA per ambulance. Larger hardware allows larger pieces.
Is the traffic real?	The multipliers are synthetic but realistic; real OpenStreetMap roads and live traffic are listed as future work.
Did you copy other projects?	No. All code was written on the day. We compared against the three references: at most 3 identical boilerplate lines. Ideas are credited.

9. Glossary

Term	Meaning
Qubit	A quantum bit. When measured it gives 0 or 1; before that it can be in a superposition of both.
Superposition	A state that holds many bitstrings at once, each with an amplitude.
QUBO	Quadratic Unconstrained Binary Optimisation: minimise a formula with linear and pairwise terms over 0/1 variables.
Ising Hamiltonian	The same cost written with Pauli-Z operators (values ±1); the form a quantum computer can apply.
Pauli Z / ZZ	Quantum operators measuring one qubit (Z) or the parity of two qubits (ZZ).
QAOA	Quantum Approximate Optimisation Algorithm: alternating cost and mixer layers with trainable angles γ, β.
Depth p	Number of cost + mixer layer pairs. More layers: better answers but a deeper circuit.
COBYLA	A classical, gradient-free optimiser used to tune γ and β.
CVaR (α)	Conditional Value at Risk: the average of the best α fraction of outcomes; focuses training on the best samples.
Shots	Number of times the circuit is run and measured (4,096 here).
AerSimulator	Qiskit's high-performance simulator of quantum circuits.
SamplerV2	Qiskit primitive that runs circuits and returns measured bitstrings.
Transpile	Compile a circuit into the basic gates a backend supports.
Brute force	Checking every possible answer; exact, but grows as n!.
Dijkstra	Classical algorithm for shortest paths in a road graph.
Penalty A	Weight of the constraint terms that make invalid routes expensive.
Approximation ratio	How close the average answer is to the optimum (1.0 = perfect).

Cheat sheet: numbers to remember

Item	Value
Challenge	1: Emergency Vehicle Routing
Qubits	n^2 → 9 (3 patients), 16 (4 patients)
Severity weights	1, 2, 4, 8 (Minor → Critical)
Default λ / penalty A (demo)	2 / 59.2
Ising terms (demo)	39 Pauli terms
Demo headline	Critical patient reached at 22.4 min instead of 77.2 min: **54.8 min sooner**, for +9.6 min driving
QAOA (demo, p = 2)	P(optimal) 4.63% = **24× random**; 24.7% valid samples; depth 29; 60 two-qubit gates
Best amplification	Up to **408× random** (4 patients, 16 qubits, p = 2)
Backend	qiskit_aer.AerSimulator, 4,096 shots, seed 7
Tests	22 passing
Run	uvicorn server:app --port 8077 → http://localhost:8077
Repo	github.com/rahulmj2004/minnal-q